Name of the Student……..… **Scholar Number…..……**

MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY BHOPAL
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

EXAMINATION: Mid Term Examination **MONTH and YEAR: October - November 2025**

Course: M.Tech **Semester**: I **Branch:** AC/CN/IS/AI **Subject Code**: CS AC 1166/CS CN 1154/CS IS 1154/CAI AI 1151
Subject Name: Web Search & Information Retrieval **Maximum Marks:** 20
Duration: 60 Minutes **Date:** 03/11/2025 **Time:** 09:30 AM To 10:30 AM

Note: *Answer all questions.*

Q. No.	Questions	Marks	COs
1	**(A)** Explain the structure and purpose of an inverted index used in Boolean retrieval systems. How does it overcome the limitations of a term-document incidence matrix for large document collections?	3	CO1
	(B) Consider the following document collection: D1: "When Antony found Julius Caesar dead, he cried almost to roaring; and he wept when at Philippi he found Brutus slain." D2: "Calpurnia! Peace, ho! Caesar shall forth. Cowards die many times before their deaths; The valiant never taste of death but once." D3: "I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me." D4: "Othello, the Moor of Venice, said: O, beware, my lord, of jealousy; it is the green-eyed monster which doth mock the meat it feeds on." Consider the query given below. Describe step by step how this query is processed using postings lists in an inverted index (Build inverted index for terms in the query). (a) Brutus AND Caesar AND NOT Calpurnia (b) Caesar OR Calpurnia (c) (Antony OR Brutus) AND NOT Othello	2	
2	**(A)** Why are stop-words like the, of, and often removed during indexing? Suppose your corpus has 100000 tokens and the following frequencies: Stop-words = 60000 tokens, Content words = 40000 tokens. If removing stop-words reduces index size by 70 %, estimate the new average posting-list length per term assuming the dictionary has 10000 terms.	2	CO2
	(B) A postings list for the term **"machine"** contains the following 20 document IDs: [3, 7, 9, 11, 14, 17, 20, 25, 28, 31, 35, 40, 44, 49, 53, 60, 65, 70, 78, 81] Another term **"learning"** has the postings list: [25, 49, 60, 81] (a) If standard postings list intersection is used (no skips), compute the total number of comparisons needed to find the intersection. (b) Now assume skip pointers are inserted every √P positions (where P=20). (i) Determine how many skip pointers will be created. (ii) Using the skip pointers, estimate how many postings comparisons can be avoided, and compute the new total number of comparisons.	3	
3	**(A)** Explain analytically how a permuterm index enables efficient processing of both prefix (auto*) and suffix (*tion) queries. Also state one key limitation in terms of space or update cost.	2	CO2
	(B) **(i)** Using the Levenshtein algorithm, Compute the edit distance between the words kitten and sitting. Backtrack and show what operations are performed. **(ii)** If your retrieval system allows correction within edit distance ≤ 2, determine whether kitten → sitting would be accepted.	3	
4	**(A)** For k = 3, list all 3-grams (use $ boundaries) for the words **"information"** and **"infomation".** Compute Jaccard Coefficient. If the acceptance threshold is 0.7, determine whether these words will be accepted as a tolerant match. Explain how changing k from 3 to 4 affects Jaccard similarity and tolerance to spelling errors.	2	CO2
	(B) **(i)** A search engine returns 20 documents, out of which 12 are relevant to the user's query. The total number of relevant documents in the collection is 30. Compute precision and recall for this query. **(ii)** If the system is tuned to retrieve more documents (40 total, 18 relevant), compute the new precision and recall. Discuss the trade-off between precision and recall observed.	3	

**********END**********

Solution

Q1 (A)

An inverted index stores a mapping from each term to the list of documents containing that term. It has two main parts:

- Dictionary: all unique terms in the collection.
- Postings list: for each term, a list of document IDs (and optionally frequencies or positions).

It enables fast Boolean retrieval by quickly locating documents for any term and combining postings lists using AND/OR operations.

Unlike a term-document incidence matrix, which is large and mostly zeros, the inverted index stores only existing term–document pairs—making it much more space-efficient and faster for large document collections.

Purpose

The purpose of the inverted index is to support **fast term-based lookups** and **Boolean query operations** (e.g., AND, OR, NOT).

For example:

- Query: "AI" AND "system"

  → Intersect postings lists for "AI" and "system" to find common documents.

Thus, the inverted index allows:

- **Efficient retrieval:** Only relevant postings lists are scanned.
- **Scalability:** Handles millions of documents.
- **Compact storage:** Only stores necessary mappings instead of entire term-document matrices.

Limitations of Term-Document Incidence Matrix

A **term-document incidence matrix** is a binary matrix where:

- Rows represent terms.
- Columns represent documents.
- A cell (i, j) = 1 if term *i* occurs in document *j*, else 0.

Problems:

1. **High Space Complexity:**

   For large collections with millions of terms and documents, most cells are zero (sparse matrix). ⇒ Wastes enormous storage space.

2. **Inefficient Query Processing:**

   Boolean operations (AND, OR) must scan full columns for all documents, even if few contain the term.

3. **Poor Scalability:**

   Adding new documents or updating terms requires modifying large matrix portions.

(B)

- **Antony** → {D1}
- **Brutus** → {D1, D3}
- **Caesar** → {D1, D2, D3}
- **Calpurnia** → {D2}
- **Othello** → {D4}

  Universal set **U** = {D1, D2, D3, D4}

(a) Brutus AND Caesar AND NOT Calpurnia

1. Brutus ∩ Caesar = {D1, D3} ∩ {D1, D2, D3} = **{D1, D3}**
2. NOT Calpurnia = U − {D2} = **{D1, D3, D4}**
3. {D1, D3} ∩ {D1, D3, D4} = **{D1, D3}**

Answer: {D1, D3}

(b) Caesar OR Calpurnia

{D1, D2, D3} ∪ {D2} = **{D1, D2, D3}**

Answer: {D1, D2, D3}

(c) (Antony OR Brutus) AND NOT Othello

1. Antony ∪ Brutus = {D1} ∪ {D1, D3} = **{D1, D3}**
2. NOT Othello = U − {D4} = **{D1, D2, D3}**
3. {D1, D3} ∩ {D1, D2, D3} = **{D1, D3}**

Answer: {D1, D3}

Q2 (A)

Reason for removing stop-words:

Stop-words such as *the, of, and* are extremely frequent but carry little meaning. Removing them:

- Reduces index size,
- Speeds up retrieval, and
- Focuses the index on content-bearing words that distinguish documents.

Given:

- Total tokens = 100,000
- Stop-words = 60,000
- Content words = 40,000
- Index size reduced by 70% after removing stop-words
- Dictionary size = 10,000 terms

Calculation:
If index size reduces by 70%, only **30%** remains.

Hence, new effective postings = 0.30 × 100,000 = **30,000**

Average posting-list length per term

= 30,000 / 10,000

= **3 postings per term**

Removing stop-words reduces redundant entries and compresses the index by 70%; the new average posting-list length ≈ **3 per term.**

(B)

Given

machine: A = [3, 7, 9, 11, 14, 17, 20, 25, 28, 31, 35, 40, 44, 49, 53, 60, 65, 70, 78, 81] (P=20)

learning: B = [25, 49, 60, 81]

(a) Standard intersection (no skips)

Walk both lists; each step compares current docIDs.

Comparisons made = $|A| + |B| - |A \cap B| = 20 + 4 - 4 =$ **20**.

(b) With skip pointers every √P positions (P=20)

$\sqrt{P} \approx 4.47 \rightarrow$ use spacing ≈ 4.

(i) Number of skip pointers
Place a skip about every 4 postings along A → from positions 1→5, 5→9, 9→13, 13→17 ⇒ **4 skips** (≈ √P).

(ii) Comparisons avoided & new total
Skips used during intersection:

- Jump 3 → 14 (bypasses 7, 9, 11) ⇒ avoids **3** element-by-element comparisons.
- Jump 28 → 44 (bypasses 31, 35, 40) ⇒ avoids **3** more.

Total avoided ≈ **6** comparisons.

So new total ≈ **20 − 6 = 14** comparisons (counting only element-by-element comparisons).

If you also count the 4 skip-check comparisons needed to decide those jumps, operational total ≈ **18**.

<u>Q3 (A)</u>

A **permuterm index** supports wildcard queries (prefix, suffix, or infix) by storing **all rotations** of each term followed by a special end symbol $.
Example:
For **"machine"**, store
machine$, achine$m, chine$ma, hine$mac, ine$mach, ne$machi, e$machin, $machine.

Prefix query (auto*)

Query auto* is expanded to auto$.
All rotations beginning with auto$ (like auto$motive, auto$graph) are retrieved directly using **alphabetical range search** — no full scan is needed.

Suffix query (*tion)

Query *tion is rewritten as tion$ and rotated to tion$*.
Matching terms (e.g., action, nation) are found by prefix-matching rotations beginning with tion$.

Thus, **both prefix and suffix queries** become **simple prefix lookups** over the permuted dictionary.

Key Limitation

A permuterm index greatly increases space and update cost —
each word with length *L* generates *(L + 1)* rotations, making the index **large and costly to update** when new terms are added.

Permuterm index enables prefix/suffix wildcard queries via rotated term lookups, but it suffers from **high storage and maintenance overhead.**

(B)

(i) Levenshtein (edit) distance: kitten → sitting

Let S1 = "kitten", S2 = "sitting".
Cost of insert, delete is considered as 1
Cost of substitute is considered as 2

	Ø	**S**	**I**	**T**	**T**	**I**	**N**	**G**
Ø	0	1	2	3	4	5	6	7
K	1	2	3	4	5	6	7	8
I	2	3	2	3	4	5	6	7
T	3	4	3	2	3	4	5	6
T	4	5	4	3	2	3	4	5
E	5	6	5	4	3	4	5	6
N	6	7	6	5	4	3	4	5

2 Replace [Substitutions (K→S, E→I)] and 1 Insert [Insertion (G)]

(ii) Acceptance with threshold ≤ 2

Since the minimum distance is **3**, and the system accepts only edits with **distance ≤ 2**,
kitten → sitting is NOT accepted.

Q4) (A)

Here's a tight, exam-style solution.

3-grams with $ boundaries (k = 3)

(Use two $ at each end for k−1 padding.)

- **information** → $$i, $in, inf, nfo, for, orm, rma, mat, ati, tio, ion, on$, n$$
  (set size = 13)
- **infomation** → $$i, $in, inf, nfo, fom, oma, mat, ati, tio, ion, on$, n$$
  (set size = 12)

Jaccard Coefficient

- Intersection (10): $$i, $in, inf, nfo, mat, ati, tio, ion, on$, n$$
- Union size = 15
- **Jaccard = 10 / 15 = 2/3 ≈ 0.667**

Decision @ threshold 0.7: 0.667 < 0.7 ⇒ **Not accepted** as a tolerant match.

Effect of changing k from 3 → 4

Larger k makes n-grams more specific; a single character error disrupts **more** k-grams, so Jaccard similarity **drops** and tolerance to spelling errors **decreases**.

(For illustration with k=4:
|information| set size 14; |infomation| set size 13; intersection 10; union 17 → **J ≈ 0.588**, lower than with k=3.)

(B)

(i)

- Documents retrieved = 20
- Relevant retrieved = 12
- Total relevant in collection = 30

Precision (P) = Relevant retrieved / Retrieved = 12 / 20 = **0.6**
Recall (R) = Relevant retrieved / Total relevant = 12 / 30 = **0.4**

Precision = 0.6, **Recall = 0.4**

(ii) After tuning (retrieves 40 docs, 18 relevant)

Precision (P') = 18 / 40 = **0.45**
Recall (R') = 18 / 30 = **0.6**

Precision = 0.45, **Recall = 0.6**

<u>Trade-off</u>

As more documents are retrieved, **recall increases** (more relevant items found), but **precision drops** (more non-relevant items included).

Hence, retrieval systems must balance between **high precision (quality)** and **high recall (completeness)** depending on the application.